

01 — Transaction Basics

Table of Contents

1. [Learning Objectives](#)
 2. [What Is a Transaction?](#)
 3. [ACID Properties](#)
 4. [Transaction Lifecycle](#)
 5. [BEGIN, COMMIT, ROLLBACK](#)
 6. [SAVEPOINT and Partial Rollback](#)
 7. [Implicit vs Explicit Transactions](#)
 8. [Transaction Isolation Overview](#)
 9. [Atomicity Guarantees in PostgreSQL](#)
 10. [Error Handling Inside Transactions](#)
 11. [Common Mistakes](#)
 12. [Best Practices](#)
 13. [Performance Considerations](#)
 14. [Interview Questions and Answers](#)
 15. [Exercises and Solutions](#)
 16. [Production Troubleshooting Scenarios](#)
 17. [Cross-References](#)
-

Learning Objectives

After studying this file you will be able to:

- Explain what a database transaction is and why it exists
 - Describe each of the four ACID properties with concrete PostgreSQL examples
 - Write correct BEGIN / COMMIT / ROLLBACK blocks
 - Use SAVEPOINT to create partial-rollback checkpoints within a transaction
 - Distinguish auto-commit from explicit transactions
 - Understand how PostgreSQL enforces atomicity through WAL and MVCC
 - Identify the most common transaction anti-patterns in production code
-

What Is a Transaction?

A **transaction** is a logical unit of work that groups one or more SQL statements so they succeed or fail together. The database guarantees that either **all** changes in the group are made permanent, or **none** are — there is no partial state visible to other sessions.

Why Transactions Matter

Consider a bank transfer:

```
UPDATE accounts SET balance = balance - 500 WHERE id = 1; -- debit
UPDATE accounts SET balance = balance + 500 WHERE id = 2; -- credit
```

If the server crashes between the two statements, account 1 loses \$500 and account 2 gains nothing. Wrapping the pair in a transaction prevents this: if the second UPDATE never commits, neither does the first.

ACID Properties

Property	Meaning
Atomicity	All operations commit together or all are rolled back.
Consistency	DB moves from one valid state to another; constraints hold.
Isolation	Concurrent transactions behave as if they run serially.
Durability	Committed data survives crashes (WAL on disk before ack).

Atomicity in Detail

PostgreSQL implements atomicity through:

- 1. **Write-Ahead Logging (WAL)** — every change is recorded in WAL before the data page is modified. On crash, the recovery process replays committed WAL records and discards uncommitted ones.
- 2. **MVCC** — uncommitted tuple versions are visible only to their own transaction; a crash leaves them invisible to everyone.

Consistency in Detail

Consistency means the database enforces all declared constraints (PRIMARY KEY, FOREIGN KEY, CHECK, NOT NULL, UNIQUE) at commit time (or immediately for non-deferred constraints). Application logic is responsible for business-rule consistency.

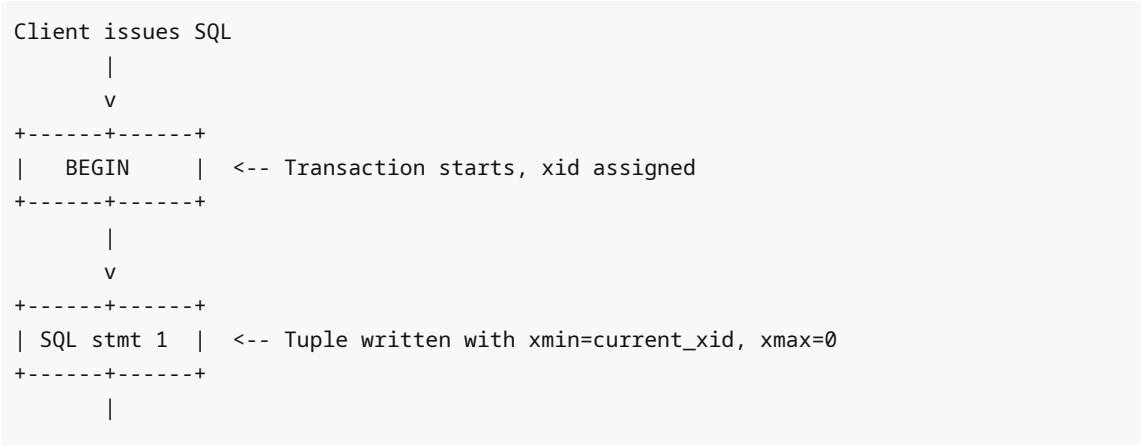
Isolation in Detail

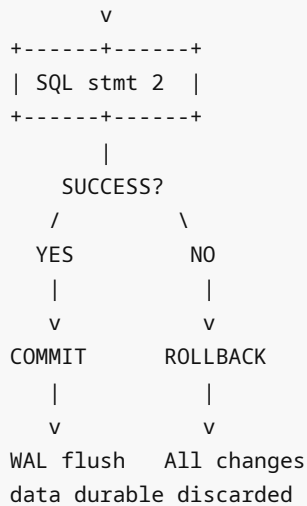
Isolation is governed by the configured **isolation level** (see 03_isolation_levels.md). PostgreSQL defaults to `READ COMMITTED` .

Durability in Detail

When `COMMIT` returns successfully, the WAL record for that transaction has been flushed to disk (via `fsync` or equivalent). The data pages themselves may still be in `shared_buffers` and written to disk later by the `bgwriter` or `checkpointer` — but recovery can always reconstruct them from WAL.

Transaction Lifecycle





BEGIN, COMMIT, ROLLBACK

Basic Syntax

```
BEGIN;                                -- or BEGIN WORK; or START TRANSACTION;
  UPDATE accounts SET balance = balance - 500 WHERE id = 1;
  UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;                                -- or COMMIT WORK; or END;
```

To abort:

```
BEGIN;
  DELETE FROM orders WHERE created_at < '2020-01-01';
ROLLBACK;                                -- nothing deleted
```

Transaction Modes

```
-- Set isolation level and access mode at open time
BEGIN ISOLATION LEVEL SERIALIZABLE READ ONLY;
  SELECT * FROM reports WHERE period = '2024-Q1';
COMMIT;

-- Or use SET TRANSACTION (must be first statement after BEGIN)
BEGIN;
  SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
  SET TRANSACTION READ ONLY;
  SELECT ...;
COMMIT;
```

Checking Transaction State

```
-- See current transaction id
SELECT txid_current();

-- See current snapshot
SELECT txid_current_snapshot();

-- Is this transaction id in progress?
SELECT txid_status(12345);  -- 'in progress', 'committed', 'aborted', or NULL
```

SAVEPOINT and Partial Rollback

A SAVEPOINT marks a point within a transaction to which you can roll back without aborting the entire transaction.

Syntax

```
BEGIN;
  INSERT INTO orders (customer_id, total) VALUES (1, 100.00);

  SAVEPOINT before_items;

  INSERT INTO order_items (order_id, product_id, qty) VALUES (1001, 99, 2);
  -- Suppose we detect an error in application logic:
  ROLLBACK TO SAVEPOINT before_items;

  -- order_items insert was undone, but orders insert still stands
  INSERT INTO order_items (order_id, product_id, qty) VALUES (1001, 50, 2);

  COMMIT;
```

RELEASE SAVEPOINT

```
BEGIN;
  SAVEPOINT sp1;
  UPDATE inventory SET qty = qty - 1 WHERE sku = 'ABC';
  SAVEPOINT sp2;
  UPDATE inventory SET qty = qty - 1 WHERE sku = 'DEF';
  RELEASE SAVEPOINT sp2;  -- removes sp2; does NOT undo its changes
  COMMIT;
```

Savepoint Rules

- Savepoints nest; `ROLLBACK TO sp1` removes all savepoints created after sp1.
- You cannot roll back past the `BEGIN`.
- After `ROLLBACK TO SAVEPOINT`, the savepoint itself still exists and can be rolled back to again.
- In PL/pgSQL, unhandled exceptions automatically roll back to the last savepoint (subtransaction).

ASCII Diagram: Savepoint Stack

```

BEGIN
|
+-- INSERT orders          (txn still active)
|
SAVEPOINT sp1
|
+-- INSERT order_items     (txn still active)
|
SAVEPOINT sp2
|
+-- UPDATE inventory       (txn still active)
|
ROLLBACK TO sp2           --> undoes UPDATE inventory; sp2 still exists
|
+-- UPDATE inventory v2    (different values)
|
RELEASE sp2               --> sp2 removed, changes kept
|
COMMIT                    --> all surviving changes made durable

```

Implicit vs Explicit Transactions

Auto-Commit Mode

By default, every SQL statement that is not inside an explicit `BEGIN` block runs in its own implicit transaction that is immediately committed on success or rolled back on error. This is called **auto-commit** mode.

```

-- Each statement is its own transaction:
INSERT INTO logs (msg) VALUES ('event 1'); -- auto-committed
INSERT INTO logs (msg) VALUES ('event 2'); -- auto-committed separately

```

Explicit Transactions

```

-- Both inserts succeed or both fail:
BEGIN;
  INSERT INTO logs (msg) VALUES ('event 1');
  INSERT INTO logs (msg) VALUES ('event 2');
COMMIT;

```

Client Library Behavior

Library	Default Mode	How to disable auto-commit
psycopg2	auto-commit OFF	Use <code>conn.autocommit = True</code>
psycopg3	auto-commit OFF	Use <code>conn.autocommit = True</code>
asyncpg	auto-commit ON	Use <code>async</code> with <code>conn.transaction()</code>
JDBC	auto-commit ON	<code>conn.setAutoCommit(false)</code>

node-postgres	auto-commit ON	Wrap with BEGIN/COMMIT
---------------	----------------	------------------------

Atomicity Guarantees in PostgreSQL

WAL-Based Atomicity

1. Before any heap page is modified, a WAL record describing the change is written to the WAL buffer.
2. At COMMIT, the WAL buffer for the transaction is flushed to disk (`pg_wal/` directory).
3. Only after the WAL flush does the client receive the success acknowledgement.
4. If the server crashes before step 2, the change is gone — as if it never happened.
5. If the server crashes after step 2, recovery replays the WAL and makes the change permanent.

MVCC-Based Atomicity

Each tuple (row version) carries:

- `xmin` : the transaction that created this tuple version
- `xmax` : the transaction that deleted/updated this tuple version (0 = still live)

An uncommitted tuple (xmin's transaction is still in progress) is **invisible to all other transactions**. If the transaction aborts, the tuple simply remains with its xmin marked as aborted in the commit log (`pg_clog` / `pg_xact`), and VACUUM will eventually remove it. No undo log is needed.

Error Handling Inside Transactions

Transaction Aborted State

After any error inside a transaction, PostgreSQL enters an **aborted** state. All subsequent commands fail until you ROLLBACK.

```
BEGIN;
  INSERT INTO t VALUES (1);
  INSERT INTO t VALUES (1); -- duplicate key ERROR
  -- Now in aborted state:
  INSERT INTO t VALUES (2); -- ERROR: current transaction is aborted,
                             --      commands ignored until end of transaction
block
ROLLBACK;
```

Using SAVEPOINT for Error Recovery

```
BEGIN;
  INSERT INTO t VALUES (1);

  SAVEPOINT sp;
  INSERT INTO t VALUES (1); -- ERROR: duplicate key
  ROLLBACK TO SAVEPOINT sp; -- back to valid state

  INSERT INTO t VALUES (2); -- succeeds
COMMIT;
```

PL/pgSQL Exception Handling

```
DO $$
BEGIN
    INSERT INTO t VALUES (1);
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'Duplicate, skipping.';
END;
$$;
```

Internally, each PL/pgSQL `BEGIN ... EXCEPTION` block creates a subtransaction (savepoint) automatically.

Common Mistakes

1. Long-Running Transactions

```
-- BAD: opens a transaction, then blocks for user input
BEGIN;
SELECT * FROM cart WHERE session_id = $1;
-- ... application waits for user to click "checkout" ...
UPDATE inventory SET qty = qty - 1 WHERE ...;
COMMIT;
```

Long-running transactions hold row locks, bloat `pg_xact`, and prevent `VACUUM` from reclaiming dead tuples.

2. Forgetting ROLLBACK on Error

```
-- BAD: in a loop, ignoring errors
FOR rec IN SELECT ... LOOP
    BEGIN;
    INSERT INTO ...;
    -- if exception occurs, connection is left in aborted state
END LOOP;
```

3. DDL Inside Long Transactions

DDL statements (`ALTER TABLE`, `CREATE INDEX`) acquire aggressive locks. Placing them inside long transactions blocks the entire table.

4. Nested BEGIN

PostgreSQL does not support true nested transactions. A second `BEGIN` issues a warning and is ignored.

```
BEGIN;
    BEGIN;           -- WARNING: there is already a transaction in progress
    INSERT ...;
COMMIT;
```

5. Relying on Auto-Commit for Multi-Statement Business Logic

```
-- BAD: no BEGIN/COMMIT — each statement is independent
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
-- A crash between these leaves the DB inconsistent
```

Best Practices

1. **Keep transactions short** — begin them as late as possible, commit as early as possible.
2. **Always handle errors** — wrap transactions in try/catch and ROLLBACK on failure.
3. **Use SAVEPOINT for recoverable sub-operations** in batch processing.
4. **Avoid user interaction inside a transaction** — commit before prompting the user.
5. **Set lock_timeout and statement_timeout** to prevent runaway transactions.
6. **Monitor long transactions** via `pg_stat_activity` — alert on anything over 5 minutes.
7. **Use READ ONLY** for reporting queries to avoid unnecessary transaction overhead.
8. **Prefer explicit BEGIN/COMMIT** over auto-commit for multi-statement operations.

```
-- Set timeouts per session
SET lock_timeout = '5s';
SET statement_timeout = '30s';
```

Performance Considerations

Transaction Overhead

Each transaction incurs:

- XID assignment (cheap but finite — XID wraparound is a real risk)
- WAL flush at COMMIT (most expensive — involves `fsync`)
- Lock manager entries

Batching for Throughput

```
-- BAD: 10,000 individual transactions
INSERT INTO events VALUES (...); -- repeated 10,000 times

-- GOOD: one transaction
BEGIN;
INSERT INTO events VALUES (...), -- batch of 10,000 rows
                        (...),
                        ...;
COMMIT;
```

synchronous_commit

```
-- Trade durability for speed (risky – only for non-critical data)
SET synchronous_commit = off;
INSERT INTO metrics VALUES (...); -- WAL write is asynchronous
```

With `synchronous_commit = off`, COMMIT returns before WAL is flushed. Up to `wal_writer_delay` (default 200ms) of commits can be lost on a crash, but the DB remains consistent.

Connection Pooling and Transaction Modes

PgBouncer in **transaction mode** releases connections back to the pool after each COMMIT/ROLLBACK — ideal for short-lived transactions. Session-level features (prepared statements, SET, advisory locks) do not work in transaction mode.

Interview Questions and Answers

Q1. What are the four ACID properties and how does PostgreSQL implement each?

A: Atomicity via WAL (crash recovery discards uncommitted WAL records) and MVCC (uncommitted tuples invisible to others). Consistency via constraint enforcement at commit time. Isolation via MVCC and lock management, configurable at four levels. Durability via WAL fsync before COMMIT acknowledgement.

Q2. What happens if you issue a second BEGIN inside an open transaction?

A: PostgreSQL issues a WARNING ("there is already a transaction in progress") and ignores the second BEGIN. The transaction continues normally.

Q3. What is the difference between ROLLBACK and ROLLBACK TO SAVEPOINT?

A: `ROLLBACK` aborts the entire transaction and releases all locks. `ROLLBACK TO SAVEPOINT` undoes all changes made after the savepoint was set, but the transaction remains open and locks are retained.

Q4. What is "aborted transaction state" and how do you recover from it?

A: After any error within an explicit transaction, PostgreSQL marks the transaction as aborted. All subsequent commands produce "ERROR: current transaction is aborted" until `ROLLBACK` is issued. Recovery: issue `ROLLBACK` (or `ROLLBACK TO SAVEPOINT` if a savepoint was set before the error).

Q5. How does PostgreSQL ensure durability without sacrificing performance?

A: WAL writes are sequential (fast), and COMMIT waits only for the WAL flush — not for data page writes. Data pages are written lazily by bgwriter and checkpoint. Recovery replays WAL to reconstruct any data pages not yet on disk.

Q6. What is synchronous_commit and when would you set it to off?

A: `synchronous_commit` controls whether COMMIT waits for WAL to be flushed to disk. Setting it to `off` allows COMMIT to return before the flush, improving throughput at the cost of up to 200ms of committed transactions being lost on a hard crash. Suitable for logging/metrics where occasional loss is acceptable but corruption is not.

Q7. What is XID wraparound and why does it matter for transactions?

A: PostgreSQL uses 32-bit transaction IDs (XIDs). After ~2 billion transactions, XIDs wrap around. Without VACUUM (which freezes old tuple XIDs), older tuples would become invisible because the new XIDs would appear "older." PostgreSQL runs emergency autovacuum when the wraparound horizon approaches.

Q8. How does SAVEPOINT work internally in PostgreSQL?

A: SAVEPOINTS are implemented as subtransactions. Each subtransaction gets its own sub-XID and WAL records. On `ROLLBACK TO SAVEPOINT`, the subtransaction's WAL is undone. The parent transaction continues.

Q9. Why are long-running transactions harmful in PostgreSQL?

A: They hold row locks (blocking writers), prevent VACUUM from removing dead tuples that are still visible to the old transaction snapshot, bloat the commit log, and can delay checkpoint completion.

Q10. What is the difference between READ COMMITTED and READ ONLY transaction modes?

A: READ COMMITTED is an isolation level — it determines when the transaction sees changes from other transactions. READ ONLY is an access mode — it prevents any data-modifying statements. They are orthogonal and can be combined.

Q11. Can DDL statements be rolled back in PostgreSQL?

A: Yes! Unlike many other databases, PostgreSQL supports transactional DDL. `CREATE TABLE`, `DROP TABLE`, `ALTER TABLE`, etc., can all be rolled back.

```
BEGIN;
CREATE TABLE test (id int);
INSERT INTO test VALUES (1);
ROLLBACK;
-- test table does not exist
```

Q12. How do you detect and terminate long-running transactions?

```
-- Find transactions open more than 5 minutes
SELECT pid, now() - xact_start AS duration, query, state
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
AND now() - xact_start > interval '5 minutes';

-- Terminate
SELECT pg_terminate_backend(pid);
```

Exercises and Solutions

Exercise 1 — Safe Bank Transfer

Write a transaction that transfers \$200 from account id=5 to account id=7. If either account has insufficient funds (balance < 200 for sender), roll back and raise a notice.

Solution:

```
DO $$
DECLARE
    v_balance numeric;
BEGIN
```

```

BEGIN; -- PL/pgSQL starts implicit transaction

SELECT balance INTO v_balance FROM accounts WHERE id = 5 FOR UPDATE;

IF v_balance < 200 THEN
    RAISE NOTICE 'Insufficient funds';
    RETURN;
END IF;

UPDATE accounts SET balance = balance - 200 WHERE id = 5;
UPDATE accounts SET balance = balance + 200 WHERE id = 7;

RAISE NOTICE 'Transfer complete.';
END;
$$;

```

Exercise 2 — Savepoint-Based Bulk Insert

Insert 5 rows into a table. If any insert fails due to a duplicate key, skip that row and continue.

Solution:

```

BEGIN;
DO $$
DECLARE
    ids int[] := ARRAY[1,2,3,2,5]; -- 2 is duplicate
    i int;
BEGIN
    FOREACH i IN ARRAY ids LOOP
        BEGIN
            INSERT INTO my_table (id) VALUES (i);
        EXCEPTION WHEN unique_violation THEN
            RAISE NOTICE 'Skipping duplicate id=%', i;
        END;
    END LOOP;
END;
$$;
COMMIT;

```

Exercise 3 — Observe Transaction State

Query `pg_stat_activity` from a second session while a transaction is open in the first session.

Solution:

```

-- Session 1:
BEGIN;
UPDATE accounts SET balance = 1000 WHERE id = 1;
-- Do NOT commit yet

-- Session 2:
SELECT pid, state, xact_start, query

```

```
FROM pg_stat_activity
WHERE state = 'idle in transaction';
```

Production Troubleshooting Scenarios

Scenario 1: "idle in transaction" Pile-Up

Symptom: Application is slow; `pg_stat_activity` shows many connections in state `idle in transaction`.

Cause: Application opened `BEGIN` but never committed (user think-time, network disconnect, application crash).

Fix:

```
-- Set idle_in_transaction_session_timeout
ALTER SYSTEM SET idle_in_transaction_session_timeout = '5min';
SELECT pg_reload_conf();

-- Immediately kill culprits
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle in transaction'
      AND now() - state_change > interval '5 minutes';
```

Scenario 2: XID Wraparound Warning

Symptom: PostgreSQL logs `WARNING: database "mydb" must be vacuumed within 177009986 transactions`.

Cause: Autovacuum is not keeping up; old frozen XIDs are approaching wraparound.

Fix:

```
-- Find tables with oldest relfrozenxid
SELECT relname, age(relfrozenxid) AS xid_age
FROM pg_class
WHERE relkind = 'r'
ORDER BY age(relfrozenxid) DESC
LIMIT 10;

-- Manually freeze
VACUUM FREEZE <table_name>;
```

Scenario 3: Transaction Causing Lock Queue

Symptom: One slow transaction is blocking 50 other connections.

Diagnosis:

```
SELECT blocking.pid AS blocking_pid,  
       blocking.query AS blocking_query,  
       blocked.pid AS blocked_pid,  
       blocked.query AS blocked_query  
FROM pg_stat_activity blocked  
JOIN pg_stat_activity blocking  
  ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))  
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0;
```

Cross-References

- [02_mvcc.md](#) — How MVCC enables non-blocking reads during open transactions
- [03_isolation_levels.md](#) — Choosing the right isolation level for your workload
- [04_locks.md](#) — Lock modes acquired by DML within transactions
- [05_deadlocks.md](#) — When two transactions block each other
- [../10_PostgreSQL_Internals/04_wal.md](#) — WAL internals that back atomicity/durability
- [../10_PostgreSQL_Internals/06_vacuum.md](#) — How VACUUM handles dead tuples from aborted transactions